

Efeitos temporários

Como um objeto lembra que algo começou, acompanha o tempo e encerra o próprio efeito — sem o PDF escrever a solução por você.

NESTA AULA

1. Entender o que um efeito temporário precisa lembrar.
2. Encontrar o lugar onde o tempo já passa no jogo.
3. Separar a responsabilidade do item e do personagem.
4. Repetir o padrão para defesa e decidir o uso repetido.

EXPLICAÇÃO E PRÁTICA INTERCALADAS

Cada explicação termina com duas questões práticas. Os exemplos completos usam outros problemas, nunca o RPG da atividade.

O ponto de partida

O personagem já melhora quando sobe de nível. Esses aumentos são permanentes: depois de somar ataque, defesa ou vida máxima, ninguém precisa lembrar de retirar o valor.

Hoje aparece outro tipo de mudança. Um efeito temporário precisa **começar agora**, continuar ativo durante um intervalo e **terminar sozinho**. Isso transforma uma soma simples em um pequeno ciclo de vida.

01 Estado e memória

O que precisa ficar guardado enquanto o efeito existe.

02 Passagem do tempo

Qual evento atualiza a duração e decide pelo encerramento.

03 Responsabilidades

O item pede; o personagem administra; o combate apenas segue.

04 Repetir o padrão

Levar a ideia de ataque para defesa e decidir o uso repetido.

O ACORDO DESTE MATERIAL

Você verá nomes de atributos, parâmetros, contratos, diagramas e exemplos análogos completos. Não verá o corpo pronto de `aplicar_bonus_ataque_temporario`, o bloco pronto de `iniciar_turno` nem a classe pronta do elixir.

Arquivos de hoje

ARQUIVO	PAPEL NESTA AULA
<code>personagem.py</code>	Guardar, atualizar e encerrar o efeito.
<code>itens.py</code>	Criar o item que solicita o efeito ao personagem.
<code>main.py</code>	Entregar o item como prêmio de uma fase.
<code>classes.py</code>	Não precisa mudar.

EXPLICAÇÃO 1

Um efeito precisa de memória

Compare duas frases: “o ataque aumentou” e “o ataque aumentou por três turnos”. A primeira descreve apenas uma alteração. A segunda também cria uma obrigação futura: alguém terá de saber **quanto retirar** e **quando retirar**.

BÔNUS PERMANENTE

Recebe uma quantidade, altera o atributo e termina o trabalho. Não existe prazo nem encerramento.

BÔNUS TEMPORÁRIO

Recebe quantidade e duração, altera o atributo e guarda informações para uma ação futura.

Os parâmetros da operação

O método pedido no RPG terá os parâmetros `quantidade` e `turnos`. Parâmetro é a informação que chega no momento da chamada:

- `quantidade` responde “qual é o tamanho do bônus?”;
- `turnos` responde “por quanto tempo ele deve continuar ativo?”.

Esses nomes não são estado permanente por si só. Quando o método termina, os valores locais desaparecem. Se o personagem precisar deles amanhã — ou no próximo turno — deverá copiá-los para atributos do próprio objeto.

Os atributos que sustentam o efeito

No `__init__`, `_bonus_ataque` representa o valor que pertence ao efeito temporário, enquanto `_turnos_bonus_ataque` representa sua duração restante. Os dois começam em zero porque um personagem recém-criado ainda não recebeu esse efeito.

INFORMAÇÃO	POR QUE GUARDAR	QUANDO MUDA
<code>ataque</code>	É o valor efetivo usado no combate.	Na aplicação e no encerramento.
<code>_bonus_ataque</code>	Permite retirar exatamente a parcela temporária.	Quando o efeito começa ou termina.
<code>_turnos_bonus_ataque</code>	Informa se o efeito está ativo e quanto falta.	Quando começa e a cada passagem de turno.

NÃO CONFUNDA VALOR ATUAL COM MEMÓRIA

Olhar apenas para `ataque` não revela qual parte veio do nível, qual veio de uma poção permanente e qual veio do elixir. O registro separado existe para que o personagem desfaça somente o que é temporário.

Exemplo parecido: uma lâmpada temporizada

Este exemplo não soma ataque e não pertence ao RPG. Ele mostra apenas a diferença entre parâmetros recebidos e estado guardado:

```
class LampadaTemporizada:
    def __init__(self):
        self.ligada = False
        self._ciclos_restantes = 0

    def ligar_por(self, ciclos):
        self.ligada = True
        self._ciclos_restantes = ciclos
```

`ciclos` chega pela chamada. `_ciclos_restantes` permanece no objeto depois que `ligar_por` termina. O RPG precisa da mesma distinção, mas trabalha com uma alteração numérica que depois deverá ser desfeita.

PRÁTICA 1 · DUAS QUESTÕES

Modelar antes de programar

Responda no código e nas anotações. O material fornece o contrato, não o corpo da solução.

QUESTÃO PRÁTICA 1

Preparar o estado

No `__init__` de `Personagem`, crie os dois registros internos discutidos na explicação: um para a parcela temporária do ataque e outro para a duração restante. Ambos devem representar “nenhum efeito ativo” quando o personagem nasce.

Depois, escreva em comentário ao lado de cada atributo qual pergunta ele responde. Não implemente ainda o encerramento.

Confira: criar Guerreiro, Mago ou inimigo continua funcionando; os novos atributos pertencem a cada objeto, não à classe inteira.

QUESTÃO PRÁTICA 2

Definir a porta de entrada

Crie em `Personagem` o método `aplicar_bonus_ataque_temporario(self, quantidade, turnos)`. Ele deve iniciar o efeito imediatamente e deixar no objeto tudo que será necessário para encerrá-lo depois.

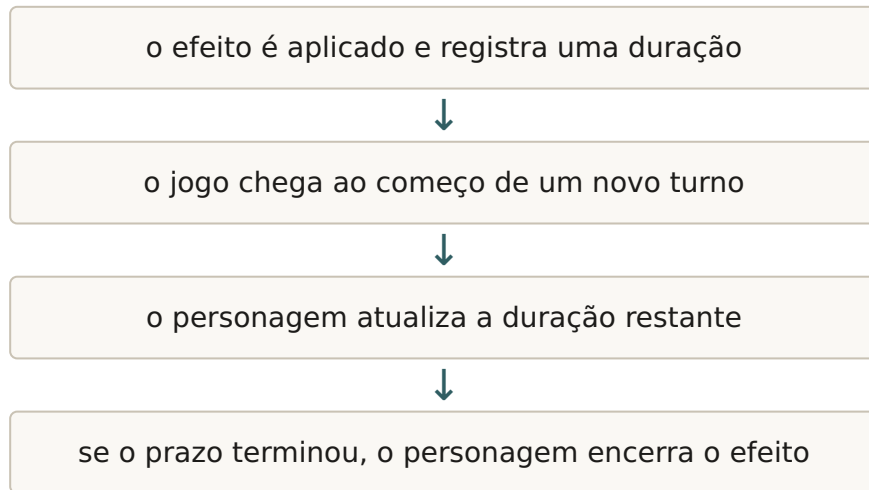
Antes de escrever o corpo, responda: quais três atributos do personagem precisam mudar quando o efeito começa? Qual deles é usado pelo combate e quais dois existem para o futuro?

Teste de comportamento: crie um Guerreiro, registre o ataque antes, aplique um bônus escolhido por você e inspecione ataque, parcela guardada e duração guardada. Explique por que os três valores são coerentes.

EXPLICAÇÃO 2

O tempo precisa de um lugar para passar

Guardar uma duração não faz o tempo andar. Um contador só muda quando algum trecho do programa é executado. Portanto, todo efeito temporário precisa de um **evento recorrente**: um ponto que o jogo já alcança uma vez por turno.



O RPG já possui esse evento: `iniciar_turno`. Ele é chamado pelo combate antes da ação do jogador e também antes da ação do inimigo. Hoje ele continua zerando `defendendo` e ganha a responsabilidade de atualizar efeitos que pertencem ao personagem.

POR QUE NÃO COLOCAR ISSO EM COMBATE?

O combate conhece a ordem dos turnos. Ele não deveria conhecer os detalhes de cada efeito. Ao chamar `iniciar_turno`, ele avisa que o tempo avançou; o próprio personagem decide o que isso significa para seu estado.

Exemplo parecido: a lâmpada aprende a contar

```
class LampadaTemporizada:
    def __init__(self):
        self.ligada = False
        self._ciclos_restantes = 0

    def ligar_por(self, ciclos):
        self.ligada = True
        self._ciclos_restantes = ciclos

    def passar_ciclo(self):
        if self._ciclos_restantes > 0:
            self._ciclos_restantes -= 1

        if self._ciclos_restantes == 0:
            self.ligada = False
```

Observe o domínio: a lâmpada encerra um estado booleano; o personagem terá de desfazer uma alteração numérica. Por isso este código ajuda a pensar, mas não pode ser copiado como resposta.

O significado de “três turnos”

No jogo atual, o jogador usa o elixir **depois** de `iniciar_turno`. O primeiro desconto acontece apenas no começo do turno seguinte. Nesta aula, “duração 3” significará: **o efeito termina no começo do terceiro turno seguinte ao uso.**

MOMENTO	O QUE ANALISAR
Turno do uso	O efeito começa depois da atualização daquele turno.
1º turno seguinte	A duração sofre a primeira atualização.
2º turno seguinte	O efeito ainda não chegou ao momento combinado de encerramento.
3º turno seguinte	O prazo se encerra antes da nova ação.

PRÁTICA 2 · DUAS QUESTÕES

Atualizar e encerrar

Preserve o comportamento antigo de `iniciar_turno` enquanto acrescenta o novo.

QUESTÃO PRÁTICA 3

Ligar o efeito ao relógio do jogo

Amplie `iniciar_turno` para atualizar a duração do bônus de ataque somente quando houver um efeito ativo. Quando o prazo alcançar o estado de encerramento, restaure o ataque retirando apenas a parcela registrada e limpe os registros do efeito.

Não remova a linha que já cuida de `defendendo`. Explique por que limpar a parcela e a duração evita que o mesmo bônus seja desfeito outra vez.

Teste de comportamento: aplique um efeito com duração curta e chame `iniciar_turno` uma vez de cada vez, imprimindo os três estados relevantes após cada chamada. Não use o jogo inteiro para depurar esta questão.

QUESTÃO PRÁTICA 4

Construir uma linha do tempo

Escolha valores diferentes dos exemplos da explicação. Em uma tabela no comentário do seu arquivo, preveja ataque, bônus guardado e turnos restantes: antes da aplicação, logo depois dela e após cada chamada de `iniciar_turno`.

Execute o teste e compare com sua previsão. Se não coincidir, identifique se o erro está na aplicação, na contagem ou no encerramento.

Critério: o valor não pode ser retirado em toda chamada; também não pode permanecer depois que o estado indicar que o efeito terminou.

EXPLICAÇÃO 3

O item pede; o personagem administra

O Elixir de fúria existe por um instante na mão do jogador e depois sai do inventário. Se a duração ficasse guardada dentro do item, quem continuaria atualizando esse objeto depois do uso? O efeito pertence ao estado do personagem, não à mochila.

OBJETO	O QUE SABE	O QUE NÃO PRECISA SABER
ElixirDeFuria	Força e duração que oferece.	Como contar turnos ou desfazer ataque.
Personagem	Seu ataque e seus efeitos ativos.	Em qual fase o item apareceu.
Inventario	Como guardar e retirar itens.	O comportamento específico do elixir.
combate	Quando começa cada turno.	Quais atributos cada efeito altera.

Exemplo parecido: um cupom e um carrinho

```
class Cupom:
    def __init__(self, percentual):
        self.percentual = percentual

    def usar(self, carrinho):
        carrinho.aplicar_desconto(self.percentual)
```

O cupom não percorre produtos nem recalcula o total. Ele entrega uma informação ao objeto que é dono da regra. Da mesma maneira, um item do RPG deve chamar o comportamento público do personagem, em vez de alterar por fora todos os seus registros internos.

POLIMORFISMO CONTINUA FUNCIONANDO

`abrir_mochila` continua fazendo apenas `item.usar(jogador, inimigo)`. O menu não pergunta se o item cura, causa dano, aumenta para sempre ou inicia um efeito temporário.

PRÁTICA 3 · DUAS QUESTÕES

Levar o efeito ao jogo

O item deve permanecer pequeno porque a regra temporal já pertence ao personagem.

QUESTÃO PRÁTICA 5

Criar o Elixir de fúria

No `itens.py`, crie `ElixirDeFuria` como um tipo de `Item`. Ele precisa guardar um valor de força e uma duração. Seu `usar(self, dono, inimigo)` deve pedir ao dono que inicie o bônus temporário por meio do método público construído anteriormente.

Escolha valores que permitam perceber o efeito sem tornar a luta trivial. O item não deve atualizar contadores nem mexer diretamente nos atributos internos do personagem.

Teste isolado: use o item sobre um Guerreiro e inspecione o estado do personagem. Depois avance turnos chamando o comportamento do próprio personagem.

QUESTÃO PRÁTICA 6

Entregar o elixir numa fase

Importe a nova classe no `main.py` e escolha uma fase para entregá-la como prêmio. Não altere `abrir_mochila`, `Inventario` nem a classe `Fase`.

Antes de executar, explique por que essas três partes conseguem trabalhar com um item criado depois delas.

Teste no jogo: vença a fase, confira o nome na mochila, use o item e acompanhe a mudança até seu encerramento. Registre qualquer diferença entre a duração imaginada e a observada.

EXPLICAÇÃO 4

Repetir o padrão sem copiar o erro

Quando ataque funciona, defesa parece apenas uma troca de nomes. A forma realmente se repete: valor aplicado, duração restante, método de aplicação e atualização no começo do turno. Porém, cada efeito precisa de registros independentes.

EFEITO DE ATAQUE

Altera ataque e consulta somente os registros temporários de ataque.

EFEITO DE DEFESA

Altera defesa e consulta somente os registros temporários de defesa.

Se o bloco de defesa descontar a duração do ataque, os dois efeitos passam a interferir. Esse tipo de erro costuma sobreviver ao copiar e colar porque o programa continua rodando.

E se o efeito for aplicado de novo?

Uma segunda aplicação antes do término exige uma regra. Sem política explícita, é possível somar duas vezes, substituir apenas a memória mais recente e deixar uma parcela presa para sempre.

POLÍTICA POSSÍVEL	COMPORTAMENTO	DECISÃO EXIGIDA
Recusar	O novo item não produz efeito enquanto já houver um ativo.	Como avisar e se o item será consumido.
Renovar	O valor não é somado novamente; apenas o prazo recomeça.	Se valores diferentes podem substituir o anterior.
Substituir	O efeito antigo é encerrado corretamente antes do novo.	Qual ordem evita deixar restos no atributo.

LIMITE DE HOJE

Escolha e documente uma política simples. Não é necessário criar uma lista genérica de efeitos, uma nova hierarquia ou um gerenciador abstrato nesta aula.

PRÁTICA 4 • DUAS QUESTÕES

Transferir e tomar uma decisão

A última dupla verifica se você entendeu o padrão, em vez de apenas reproduzir o ataque.

QUESTÃO PRÁTICA 7

Um efeito temporário de defesa

Crie registros próprios para valor e duração de defesa, um método público que inicia esse efeito e a atualização correspondente em `iniciar_turno`. Depois crie um item de nome e valores escolhidos por você que solicite esse comportamento.

Não reutilize os registros de ataque. Ataque e defesa devem poder permanecer ativos ao mesmo tempo e terminar em momentos diferentes.

Teste combinado: aplique os dois efeitos com durações diferentes, avance o tempo e registre os quatro estados internos após cada turno. Um efeito não pode modificar o prazo nem o atributo do outro.

QUESTÃO PRÁTICA 8

Definir o uso repetido

Escolha uma das políticas discutidas: recusar, renovar ou substituir. Escreva duas frases explicando a regra para o jogador e implemente-a primeiro apenas no bônus de ataque.

Crie um teste que aplique um segundo efeito antes do primeiro terminar. Antes de rodar, anote qual deve ser o ataque final depois que todo prazo acabar.

Critério: ao final, o ataque deve conter apenas seus aumentos permanentes. Nenhuma parcela temporária pode ficar presa.

FECHAMENTO

Um bônus que sabe ir embora

A novidade de hoje não foi uma palavra nova de Python. Foi fazer um objeto sustentar uma regra ao longo do tempo: receber dados, guardar estado, reagir a um evento recorrente e limpar o que não vale mais.

- ☐ Sei explicar a diferença entre parâmetro recebido e atributo guardado.
- ☐ O personagem administra seus efeitos; o item apenas solicita.
- ☐ `iniciar_turno` preserva a defesa e atualiza os efeitos ativos.
- ☐ Ataque e defesa usam memórias independentes.
- ☐ A duração tem um significado explícito e testável.
- ☐ Existe uma política documentada para aplicar o efeito novamente.
- ☐ O jogo funciona sem mudar o menu para reconhecer itens específicos.

PRÓXIMA AULA

Com efeitos temporários funcionando, Guerreiro e Mago podem ganhar habilidades próprias. A infraestrutura de hoje será reutilizada para fazê-los jogar de formas realmente diferentes.